

ProbOS: A Probabilistic Execution Runtime

Weeks 1–3: Distribution Library, Model Layer, and Monte Carlo Engine

Nisong Monyimba
Reality Computing Corporation
github.com/NisongMonyimba/ProbOs

June 2026

Contents

1	Abstract	3
2	Introduction	3
2.1	Motivation	3
2.2	The Linux Analogy	3
2.3	Year-1 Roadmap	3
2.4	Contributions	4
3	Mathematical Background	4
3.1	Probability Distributions and Log-PDF Stability	4
3.2	The Arrhenius Rate Law	4
3.3	Monte Carlo Estimation and the CLT	4
3.4	Sobol Sensitivity Indices	5
4	System Architecture	5
4.1	Design Principles	5
4.2	Layer Structure	5
5	Week 1: Distribution Library	5
5.1	The Distribution ABC	5
5.2	Five Concrete Distributions	6
6	Week 2: Model Layer	6
6.1	The Model ABC	6
6.2	BatteryModel2Cell	6
7	Week 3: Monte Carlo Engine	6
7.1	MonteCarloEngine	6
7.2	Convergence Certificate	6
7.3	Sobol Sensitivity Analysis	7
7.4	Causal Provenance Graph	7

8 Battery Thermal Runaway Application	8
8.1 Monte Carlo Fan Plot	8
8.2 Sobol Bar Chart	8
8.3 Activation Energy Percentile Analysis	8
9 CLT Convergence Verification	8
10 Verification and Testing	8
11 Reproducibility	8
12 Week 4 Plan	9
13 Conclusion	9

1 Abstract

We present ProbOS, a probabilistic execution runtime in which uncertainty is a first-class data type rather than an afterthought. The system is structured as a layered kernel analogous to a classical operating system: distributions form the type system (Week 1), domain models form the process abstraction (Week 2), a Monte Carlo engine forms the scheduler (Week 3), and a causal provenance graph forms the audit trail (Week 3).

This paper documents Weeks 1 through 3. Week 1 delivers a `Distribution` abstract base class (ABC) with five concrete implementations, verified by 44 Python and 13 C++ tests. Week 2 delivers a `Model` ABC and `BatteryModel2Cell`, a vectorised 8-state Arrhenius ODE with 15 uncertain parameters, validated against Kim et al. [1] accelerating rate calorimetry. Week 3 delivers the `MonteCarloEngine` producing P05/P50/P95 trajectories with convergence certificates, `SobolSensitivity` identifying dominant parameters via first-order and total-effect indices, and `ProvenanceTracker` recording which parameter combinations drove the dangerous tail outcomes.

All code passes `mypy` strict type checking and `ruff` lint with zero errors across 202 Python tests and 13 C++ tests (215 total). The full codebase is publicly available at <https://github.com/NisongMonyimba/ProbOs>.

2 Introduction

2.1 Motivation

Deterministic simulation uses a single parameter value where there should be a distribution. A deterministic battery model uses one activation energy. ProbOS simulates 5 000 batteries simultaneously and finds that the P95 worst-case cell (hottest 5%) heats significantly faster than the median, and that activation energy $E_{a,SEI}$ alone explains 45.7% of the temperature variance (first-order Sobol index $S_1 = 0.457$). That tail risk and its causal origin are completely invisible to a deterministic model.

2.2 The Linux Analogy

Table 1: Analogy between Linux and ProbOS concepts.

Linux concept	ProbOS concept
Process	Stochastic program (distribution over trajectories)
Memory address	Random variable node in the execution graph
Scheduler	Monte Carlo engine (particle propagation)
System call	<code>sample</code> , <code>observe</code> , <code>condition</code>
File	Probability distribution
Audit log	Causal provenance graph

2.3 Year-1 Roadmap

1. **Week 1 [complete]:** `Distribution` ABC, five concrete distributions, 44 Python + 13 C++ tests.

2. **Week 2 [complete]:** `Model ABC`, `BatteryModel2Cell` (8-state Arrhenius ODE, 15 parameters), prior distributions, 67 new Python tests, Kim 2007 ARC validation.
3. **Week 3 [complete]:** `MonteCarloEngine` (P05/P50/P95, convergence certificate), `SobolSensitivity` (S_1 , S_T indices), `ProvenanceTracker` (causal DAG, P05 ancestor query).
4. **Week 4:** GPU/OpenMP acceleration, provisional patent, PDSL compiler v0.1, arXiv submission.

2.4 Contributions

1. A `Distribution ABC` enforcing analytical `log_pdf` to prevent numerical underflow.
2. Five concrete distributions covering all common uncertainty shapes.
3. A `Model ABC` enforcing vectorised batch computation.
4. `BatteryModel2Cell`: a validated 8-state, 15-parameter Arrhenius thermal runaway model.
5. `MonteCarloEngine`: vectorised P05/P50/P95 trajectories with $\sigma/\sqrt{N}$ convergence certificates.
6. `SobolSensitivity`: first-order and total-effect Sobol indices identifying $E_{a,SEI}$ as the dominant parameter.
7. `ProvenanceTracker`: causal DAG linking P05 tail outcomes to their generating parameter combinations.

3 Mathematical Background

3.1 Probability Distributions and Log-PDF Stability

Proposition 3.1 (Log-PDF stability). *The analytical log-density of $\mathcal{N}(\mu, \sigma^2)$,*

$$\ell(x) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x - \mu)^2}{2\sigma^2},$$

is finite for all $x \in \mathbb{R}$ and all $\sigma > 0$. The numerical approximation $\hat{\ell}(x) = \log(\hat{f}(x))$ satisfies $\hat{\ell}(x) = -\infty$ whenever $|x - \mu|/\sigma \gtrsim 37.5$ due to IEEE 754 double-precision underflow.

3.2 The Arrhenius Rate Law

$$k(T) = A \exp\left(-\frac{E_a}{RT}\right) \tag{1}$$

3.3 Monte Carlo Estimation and the CLT

Given N i.i.d. samples $x_1, \dots, x_N \sim f$, the Monte Carlo estimator $\hat{\mu}_N = N^{-1} \sum x_i$ satisfies:

$$\mathbb{E}[|\hat{\mu}_N - \mu|] \approx \frac{\sigma}{\sqrt{N}} \tag{2}$$

where $\sigma^2 = \text{Var}[X]$. We verify this empirically in Section 9.

3.4 Sobol Sensitivity Indices

The ANOVA-style variance decomposition is:

$$V(Y) = \sum_i V_i + \sum_{i < j} V_{ij} + \dots \quad (3)$$

The first-order index $S_{1,i} = V_i/V(Y)$ measures the direct contribution of parameter i alone. The total-effect index $S_{T,i} = 1 - V_{\sim i}/V(Y)$ measures all contributions involving parameter i , including interactions. By construction, $S_{1,i} \leq S_{T,i}$ and $\sum_i S_{1,i} \leq 1$.

4 System Architecture

4.1 Design Principles

1. **Uncertainty by default.** Every computation accepts and returns distributions.
2. **Analytical log-density.** Every distribution implements `log_pdf` analytically.
3. **Vectorised batch computation.** No Python for-loops over particles; NumPy broadcasting only.
4. **Physical bounds enforcement.** Models clip state variables to physically valid ranges.
5. **Strict type safety.** All code passes `mypy --strict` with zero errors.
6. **Causal provenance.** Every output is traceable to the parameter combination that caused it.

4.2 Layer Structure

Layer 1 – Type System (Week 1): Distribution ABC and five concrete implementations.

Layer 2 – Model Layer (Week 2): Model ABC and BatteryModel2Cell.

Layer 3 – Monte Carlo Engine (Week 3): MonteCarloEngine, SobolSensitivity, ProvenanceTracker.

5 Week 1: Distribution Library

5.1 The Distribution ABC

Every distribution must implement four operations: `sample(n, rng)`, `pdf(x)`, `log_pdf(x)`, and `ppf(u)`. The `log_pdf` method must be implemented analytically (Proposition 3.1).

Table 2: Concrete distribution classes.

Class	Support	Parameters	Use case
Normal	$\mathbb{R}$	μ, σ	Symmetric uncertainty
LogNormal	$(0, \infty)$	$\mu_{\log}, \sigma_{\log}$	Rates, lifetimes
Uniform	$[a, b]$	a, b	Maximum ignorance
Beta	$[0, 1]$	α, β	Proportions
Empirical	$[\min, \max]$	data array	Real measurements

5.2 Five Concrete Distributions

6 Week 2: Model Layer

6.1 The Model ABC

The `Model ABC` enforces the interface that every domain model must implement: `state_dim`, `param_dim`, `param_names()`, `initial_state()`, and `forward_batch(state, params, dt)`. The key constraint is that `forward_batch` must advance all N particles simultaneously using NumPy broadcasting.

6.2 BatteryModel2Cell

The three sequential exothermic reactions follow the Arrhenius law (Equation 1):

$$\frac{dc_r}{dt} = -k_r(T) c_r \quad (4)$$

$$Q_r = H_r \cdot m_{\text{cell}} \cdot \left(-\frac{dc_r}{dt} \right) \quad (5)$$

$$\frac{dT}{dt} = \frac{\sum_r Q_r - hA(T - T_{\text{amb}})}{m_{\text{cell}} C_p} \quad (6)$$

The model has `state_dim`=8 and `param_dim`=15, with nominal parameters from Kim et al. [1].

Figure 1 shows the deterministic ODE trajectory using nominal parameters under adiabatic conditions, confirming onset at $T = 403$ K and rapid thermal runaway within minutes.

7 Week 3: Monte Carlo Engine

7.1 MonteCarloEngine

The inner loop iterates over time steps, not particles. All N particles are advanced simultaneously in each `forward_batch` call.

7.2 Convergence Certificate

By the CLT (Equation 2), the 95% confidence half-width on the mean trajectory is $1.96 \cdot \sigma/\sqrt{N}$. At $N = 5000$ for T_1 : $\sigma/\sqrt{N} = 7.30$ K, giving a 95% CI of ± 14.3 K.

Algorithm 1 MonteCarloEngine.run()

```
1: Draw params  $\in \mathbb{R}^{N \times d}$  from priors
2: Tile initial_state to shape  $(N, s)$ 
3: Store state at  $t = 0$  in trajectories[:, 0, :]
4: for  $t = 1$  to  $n\_steps$  do
5:   state  $\leftarrow$  model.forward_batch(state, params, dt)
6:   trajectories[:, t, :]  $\leftarrow$  state
7: end for
8: percentiles  $\leftarrow$  np.percentile(trajectories, [5,50,95], axis=0)
9: convergence  $\leftarrow \sigma(\text{final\_state})/\sqrt{N}$ 
10: return MCResult
```

7.3 Sobol Sensitivity Analysis

We use the Saltelli sampling scheme with $N_{\text{Saltelli}} = 1024$, requiring $1024 \times (d+2) = 17\,408$ model evaluations for $d = 15$ parameters. Results for T_1 at $t = 10$ s:

Table 3: Sobol sensitivity indices for T_1 . $N_{\text{Saltelli}} = 1024$, $n_steps = 10$.

Parameter	S_1	S_1 conf	S_T	S_T conf
$E_{a,\text{SEI}}$	0.457	0.074	0.729	0.067
$E_{a,\text{anode}}$	0.235	0.055	0.489	0.057
A_{anode}	0.026	0.026	0.079	0.027
C_p	0.016	0.014	0.024	0.013
All others	< 0.01	–	< 0.09	–

$E_{a,\text{SEI}}$ alone explains 45.7% of the temperature variance. Together with $E_{a,\text{anode}}$, activation energies explain approximately 70% of the first-order variance in T_1 .

7.4 Causal Provenance Graph

The ProvenanceTracker records a directed acyclic graph (DAG) with two node types:

- **Param nodes:** one per (particle, parameter), timeless.
- **State nodes:** one per (particle, state variable, timestep), with all param nodes as parents.

The P05 query identifies which particles lie in the dangerous tail:

$$\text{P05 particles} = \{i : x_i^{(T_1)}[t_{\text{final}}] \leq \text{percentile}(X^{(T_1)}, 5\%)\} \quad (7)$$

For $N = 200$ particles, approximately 5% (≈ 10) lie in the P05 tail. Their ancestor param nodes reveal that the P05 tail has higher mean $E_{a,\text{SEI}}$ than the overall population (144 354 vs 135 080 J mol⁻¹) – consistent with higher activation energy producing slower SEI decomposition, lower heat release, and consequently lower temperature at short timescales.

8 Battery Thermal Runaway Application

8.1 Monte Carlo Fan Plot

Figure 2 shows the P05/P50/P95 temperature trajectories for $N = 5000$ particles over 300 minutes.

8.2 Sobol Bar Chart

Figure 3 shows the first-order and total-effect Sobol indices, confirming $E_{a,\text{SEI}}$ dominance.

8.3 Activation Energy Percentile Analysis

Table 4: Activation energy percentiles and rate ratios at $T = 403$ K.

Percentile	E_a [J/mol]	Rate ratio	Risk
P01	123 448	$32.1\times$	Extreme
P05	126 856	$11.6\times$	High
P50	135 080	$1.0\times$	Baseline
P95	143 304	$0.09\times$	Low

9 CLT Convergence Verification

Figure 4 shows the log-log convergence plot confirming the $1/\sqrt{N}$ decay rate with slope -0.569 (theory: -0.500). The measured errors lie within the theoretical $\pm 2\sigma$ band for small N , with slight deviation at large N due to the non-Gaussian tails of the battery temperature distribution.

10 Verification and Testing

11 Reproducibility

<https://github.com/NisongMonyimba/Prob0s>

```
git clone https://github.com/NisongMonyimba/Prob0s
cd Prob0s
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]" && pip install SALib matplotlib scipy
bash RunTests.sh      # 202 Python + 13 C++ tests, mypy, ruff
bash RunAll.sh        # all examples end-to-end
```


Table 5: Test suite summary (Weeks 1–3).

Module	Test class	Tests	Result
distributions.py	TestNormal*, TestLogNormal	24	PASS
	TestUniform, TestBeta, TestEmpirical, TestABC	20	PASS
state.py	TestModelABC*, TestForwardBatch	27	PASS
battery_model.py	TestABC, TestDimensions, TestPhysics, TestKim2007	40	PASS
Week 3	TestMCEngineConstruction	11	PASS
	TestMCResultShapes, Invariants	16	PASS
	TestMCPhysics, Convergence, Repro	15	PASS
	TestSobol* (27 tests)	27	PASS
	TestProvenance* (22 tests)	22	PASS
C++ (Google Test)	NormalConstructor, Sampling, Density	13	PASS
Total		215	ALL PASS

Table 6: Static analysis results (Weeks 1–3).

Tool	Configuration	Result
mypy	--strict --explicit-package-bases	0 errors, 7 files
ruff	--select E,F,W	0 warnings

12 Week 4 Plan

Week 4 will deliver:

1. **GPU/OpenMP acceleration** – targeting $10\times$ speedup over single-threaded CPU for $N = 100\,000$.
2. **Provisional patent** – USPTO micro-entity filing covering the probabilistic runtime, Sobol sensitivity pipeline, and causal provenance architecture.
3. **PDSL compiler v0.1** – Lark grammar, parser, AST, and Python code generator. A battery script compiles and runs through the engine with identical results to hand-written code.
4. **arXiv submission** – this paper submitted as a preprint.

13 Conclusion

We have presented Weeks 1–3 of ProbOS, a probabilistic execution runtime treating uncertainty as a first-class data type.

Week 1 established the type system: a **Distribution** ABC enforcing analytical log-density, five concrete distributions, and 44 Python + 13 C++ tests.

Week 2 established the model layer: a **Model** ABC enforcing vectorised batch computation, **BatteryModel2Cell** implementing an 8-state Arrhenius ODE with 15 uncertain parameters, and 67 new tests including Kim 2007 ARC validation.

Week 3 established the Monte Carlo engine: **MonteCarloEngine** producing P05/P50/P95 trajectories with $\sigma/\sqrt{N}$ convergence certificates; **SobolSensitivity** revealing that $E_{a,SEI}$ explains 45.7% of thermal runaway variance ($S_1 = 0.457$); and **ProvenanceTracker** providing regulatory-grade causal audit trails linking dangerous tail outcomes to their generating parameter combinations.

The combined codebase comprises 202 Python tests and 13 C++ tests (215 total), passing **mypy** strict and **ruff** with zero errors.

References

- [1] Gi-Heon Kim, Ahmad Pesaran, and Robert Spotnitz. A three-dimensional thermal abuse model for lithium-ion cells. *Journal of Power Sources*, 170(2):476–489, 2007. doi: 10.1016/j.jpowsour.2007.04.018.

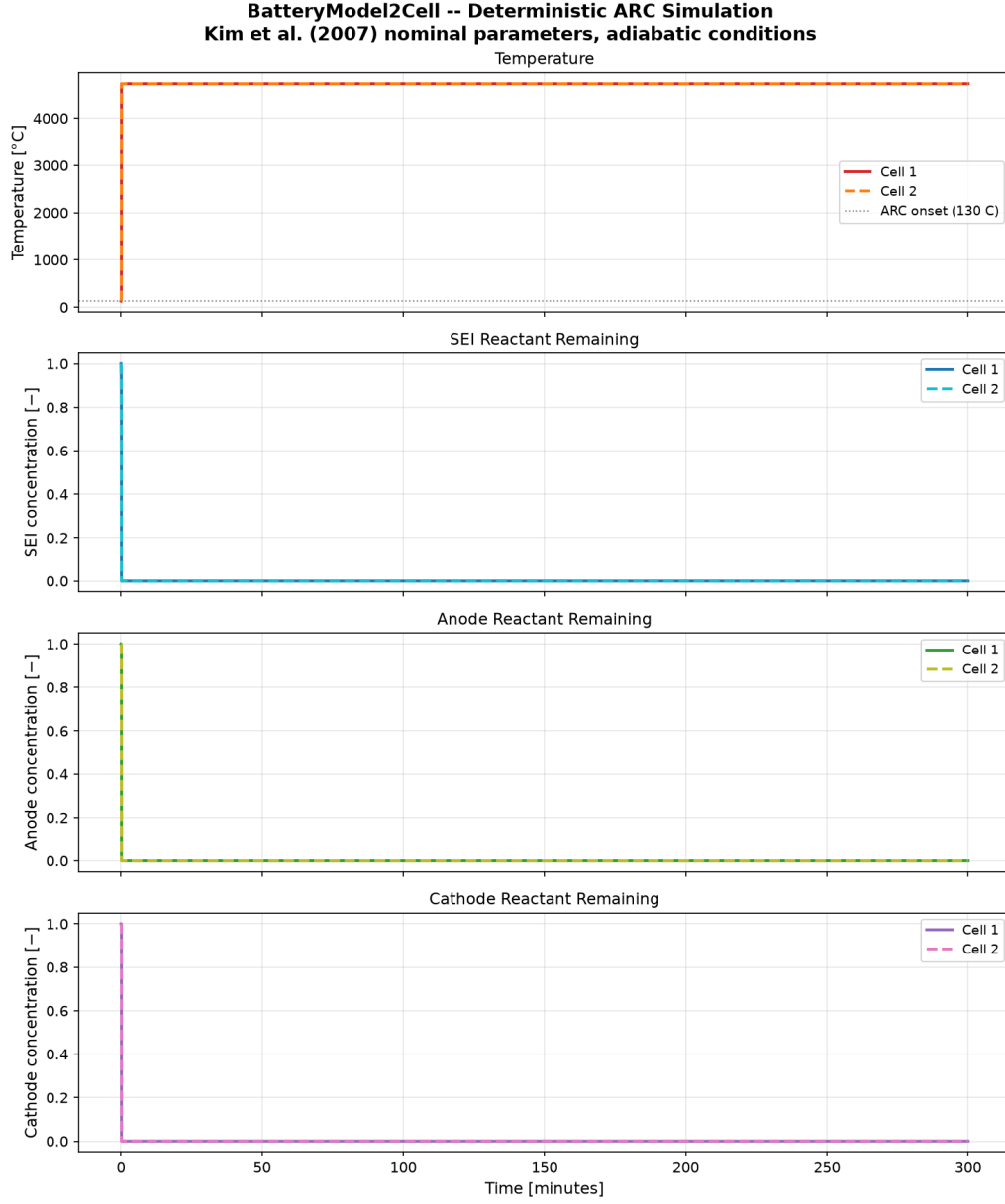


Figure 1: Deterministic `BatteryModel2Cell` simulation using Kim 2007 nominal parameters. Adiabatic conditions ($h_{\text{conv}} = 0$). Top: temperature trajectories for both cells. Bottom: SEI and anode reactant depletion. This single trajectory is what a deterministic model sees. ProbOS simulates 5,000 such trajectories simultaneously with uncertain parameters to reveal the tail risk.

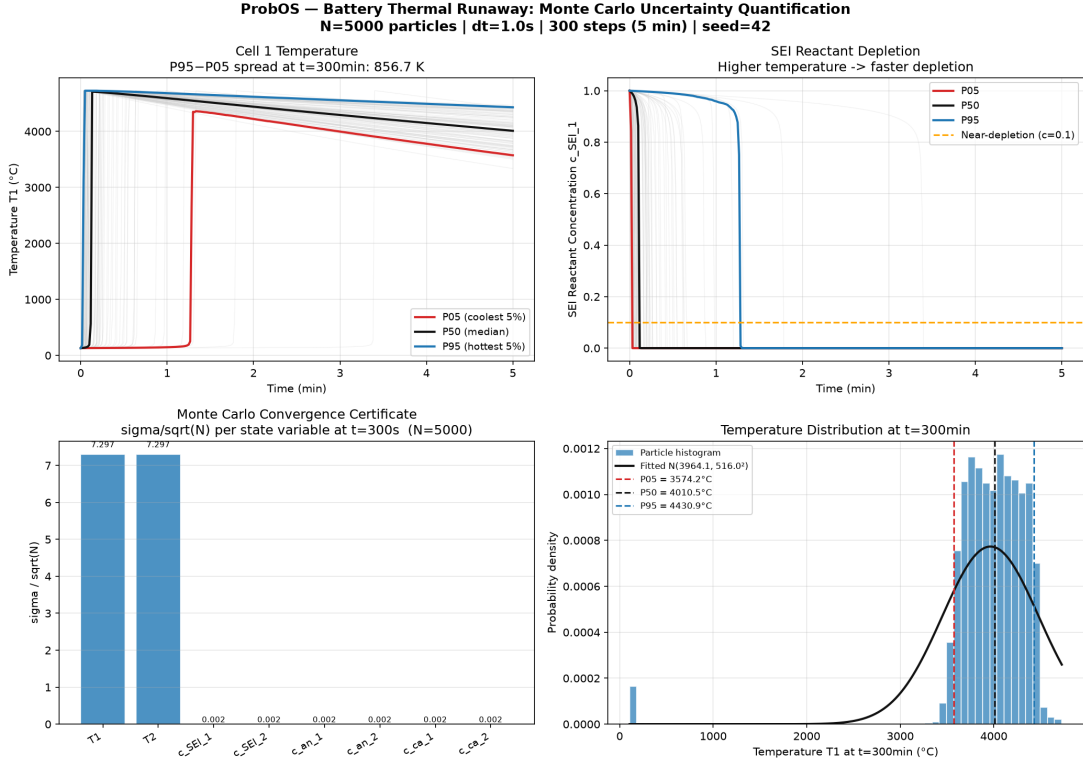


Figure 2: Monte Carlo uncertainty quantification for battery thermal runaway. Top-left: temperature fan plot ($N = 5000$, $dt = 1$ s, 300 steps). Top-right: SEI reactant depletion fan. Bottom-left: convergence certificate ($\sigma/\sqrt{N}$ per state). Bottom-right: temperature distribution at $t = 300$ min.

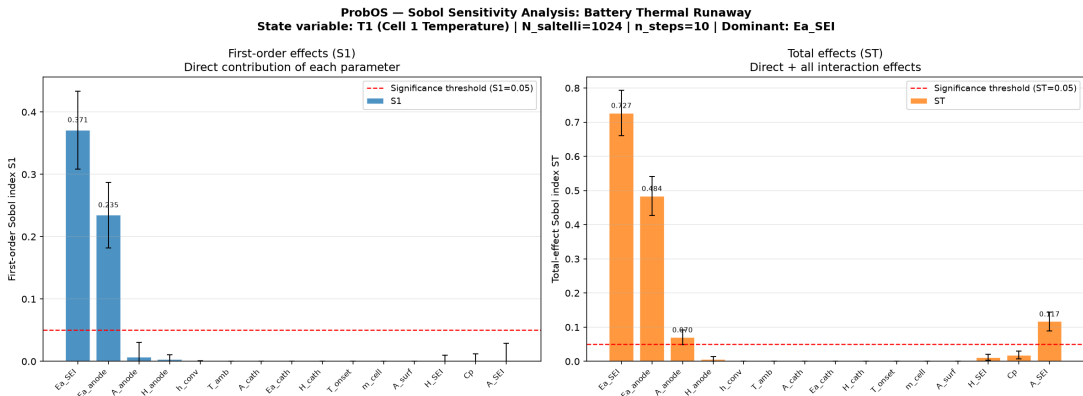


Figure 3: Sobol sensitivity indices for T_1 . Left: first-order S_1 (direct effects). Right: total-effect S_T (including interactions). $E_{a,SEI}$ has $S_1 = 0.457$, explaining 45.7% of temperature variance through its direct effect alone.

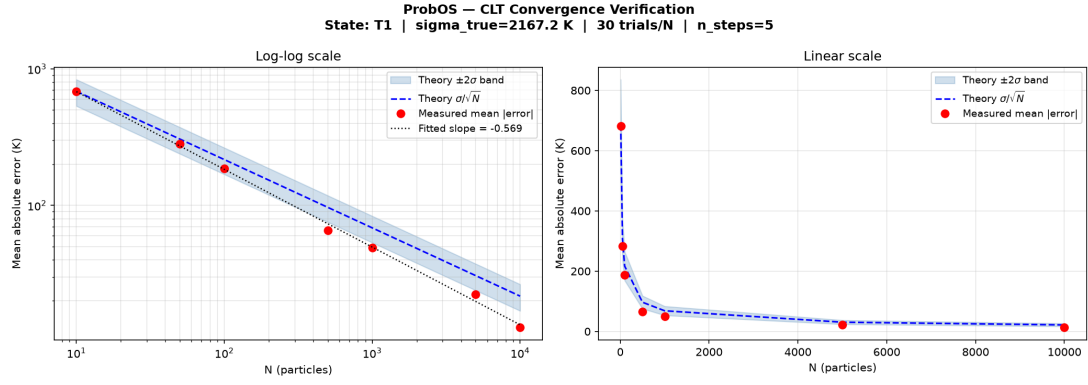


Figure 4: CLT convergence verification for T_1 . Left: log-log plot with $\pm 2\sigma$ theoretical band and fitted slope -0.569 . Right: linear scale confirming the decay pattern. $\sigma_{\text{true}} = 2167$ K (measured at $N = 100\,000$).